

INF-2300: COMPUTER COMMUNICATION

PROJECT 1

Eivind Østlyngen

EOELY

eos033@uit.no

September 9, 2022

1 Introduction

In this report, I will describe and discuss my implementation of a simple HTTP server exposing multiple endpoints. The endpoints include both web and RESTful API.

2 Technical background

2.1 HTTP

HTTP is a standardized protocol used for data exchange on the web, transferring data between the clients and the servers. The content of the data, for both responses and requests, is defined by the HTTP standard. A request is performed using HTTP methods and is responded by a status code, describing if the action was successful. [1]

2.2 JSON

JSON, Javascript object notation, is a standardized text format used to represent data. It is commonly used to send data over the web, and to display data stored in databases. [2]

3 Design & Implementation

3.1 Web endpoint

The browser endpoint supports two HTTP methods, GET and POST. When the server is active, listening to the endpoints, the user can access the URL of the website which performs GET requests to load the content of the site. These requests consists of a HTTP method and HTTP headers, describing the expected content. The response consists of a status code, headers describing the returned content such as the length and type, and the site content as the HTTP body. The status code is 200 if successful and 404 if the requested file is not found.

The POST method is simply used to append to a text file containing inputted text from the website. This is done by storing the text as the response HTTP body, allowing the server to store the text to the existing database. If the file to store the text is not created in the database does not exist, it is created and the text is added, otherwise the text is simply appended. The response is then , similarly to GET, a status code indicating that a new resource is created, headers and the complete content of the file.

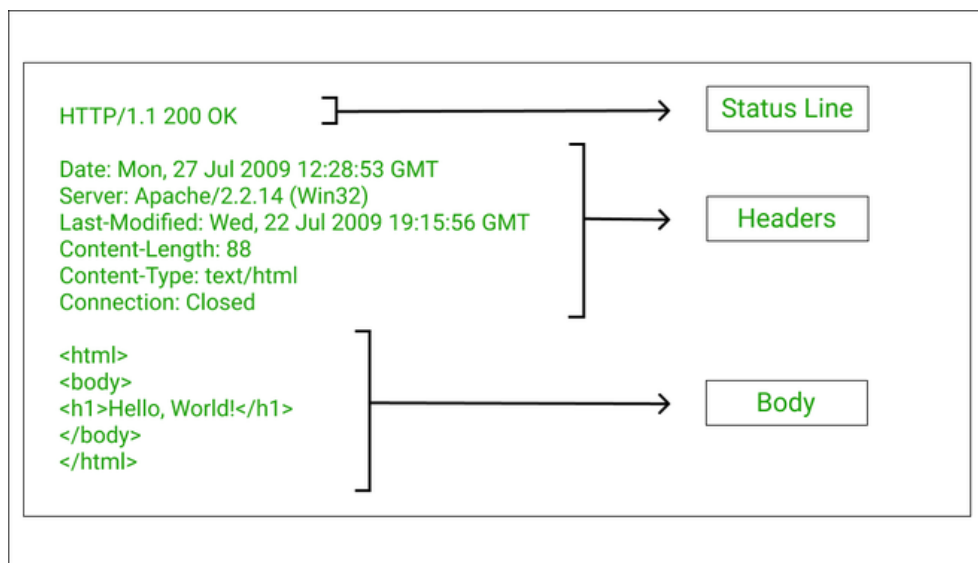


Figure 1: [3]Showcasing an example of the format of an HTTP response.

3.2 RESTful API endpoint

The purpose of the RESTful API endpoint is to support a simple message service. The endpoint should support adding, deleting and editing messages and should storing the messages between uses, which is done by performing HTTP methods using the JSON(JavaScript object notation) format and storing it in a JSON file. The format for each message is a unique ID, generated by the server, and a message from the user. The methods supported to perform these requirements are GET, POST, PUT and DELETE. Using the GET method, similarly to the web endpoint, all messages are returned, in JSON format, along with status code 200. No special headers are required in the request.

Using the POST method, a message should be appended to the database. This is done by accepting a JSON object from the user in the body with a valid message. As the ID is generated, the user cannot specify its own ID. If the user provides invalid input, no action will be performed on the server, and a bad request status will be returned to the user. If the request is valid, and the message is appended, the response will contain a status code to reflect this and the appended message with the generated ID.

The PUT method should edit an existing message within the database. This requires a request from the user containing a JSON object with both the ID of the message and the new message text. Given correct input and the existence of a message with that ID, the server will replace the message text with the response message text. The response will not contain the edited message, but will include a status code indicating success. If there are no message in the database matching the response ID, the server will respond that no message is found. The server will not create a message with that id and text. Given bad input, missing text or ID, the server will return bad request status code.

Using the DELETE method, should simply delete a message from the database. This is done by providing the ID of an existing message in JSON format. If a message exists matching the ID, the entire message is removed from the database. No response body is returned, only a status code indicating success. Similarly to POST, given bad input or non existing message the server will respond with status code describing the case.

4 Discussion

4.1 Special cases

There are many special cases to consider when implementing these endpoints, both to ensure consistent responses to the client, validity of the database data and the security of the server. To ensure consistent

responses all possible cases should return a status code, no matter the request, preventing socket hangups. To ensure the validity of the database, all input from the user is validated to contain the required fields from the client before editing the database values. Ensuring security is done by limiting what files can be accessed from the endpoints. The client does not have access to any files below the directory, preventing any traversal attacks. For the web endpoint, the POST method is limited to only the text file to prevent corrupting other files. The message API endpoint is only valid for the message database.

Given non-supported request methods such as HEAD or PIZZA simply returns bad request back to the client, and does not impact the status of the server. As mentioned above, performing POST requests to other files, such as script files, are not allowed and a unauthorized status code will be returned back to the client. The implementation for PUT request requires both a valid ID and text. If the response is missing either, the server will return bad request. Using PUT without an ID, there is no way to check the existence of a message to edit. If the client wants to append a new message with a generated ID, POST should be used instead. Without a valid text also returns bad request. The alternative here would be to erase the content, which can currently be done more controlled by specifying an empty message.

4.2 Testing

To test all the behaviour discussed earlier in the sections above, tests have been written against the database to validate the wanted response. Normal client requests have also been validated using the Postman API platform.

5 Conclusion

In conclusion the implementation meets all the specified requirements for both endpoints and works as expected, both from the tests and actually utilizing the endpoints.

6 Sources

References

- [1] Developer.mozilla.org. 2022. An overview of HTTP. [online] Available at: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview> [Accessed 2 September 2022].
- [2] Developer.mozilla.org. 2022. Working with JSON - Learn web development — MDN. [online] Available at: <https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Objects/JSON> [Accessed 2 September 2022].
- [3] Tiwari, V., 2022. State the core components of an HTTP response ? - GeeksforGeeks. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/state-the-core-components-of-an-http-response/> [Accessed 2 September 2022].